Overview of CLIP Based Approaches for Architectural Style Classification

Biagio Carpaneto, Yari Salsi, Simone Serafini

***Abstract*— Recognizing architectural styles from building images provides a focused and challenging testbed for vision–language models. This study was conducted as part of the Computer Vision course at the University of Modena and Reggio Emilia (UNIMORE). Motivated by the strong zero-shot performance of CLIP [1], we systematically benchmark several CLIP-based strategies: plain zero-shot inference, prompt ensembles, CoOp [2], linear probes on frozen CLIP features, LoRA fine-tuning [3], and nonlinear heads trained on CLIP embeddings.**

Since the choice of adaptation technique can strongly influence model performance, we ask which approach achieves the best accuracy under comparable computational power. Using consistent training protocols and evaluation metrics, our results confirm that zero-shot CLIP is a strong baseline, often rivaling task-specific pipelines. However, when limited supervision is available, the best trade-off in our experiments is obtained by coupling a frozen CLIP encoder with a shallow multi-layer perceptron trained on its embedding space. This nonlinear classifier outperforms linear probes and CoOp while preserving the simplicity and stability of frozen backbones. Moreover performing LoRA fine-tuning over the froze encoders further improved the performance.

Overall, our study clarifies when each strategy is preferable and provides practical guidance for CLIP-based classification on limited dataset. The codes are publicly available at: `https://github.com/SalsiYari/CV_team_project.git`.

I. Introduction and Motivating Work

Architectural style recognition from photographs is an image classification task where each building image is assigned to a stylistic category. In practice, this is a multi-class problem—each instance primarily belongs to one style—yet it remains challenging due to high intra-class variability (materials, viewpoints, eras) and inter-class similarity (shared visual motifs across styles).

Our goal is to establish a clear benchmark for modern classification strategies on this task, with a specific focus on contrastive vision–language models. CLIP has demonstrated strong zero-shot performance that often rivals task-specific pipelines, naturally raising the question: which adaptation technique provides the best performance for architectural styles under comparable computational budgets? Since the choice of adaptation strategy can substantially affect accuracy, we systematically analyze several CLIP-based approaches, including zero-shot inference, prompt ensembles, CoOp prompt learning, linear probes on frozen CLIP features, LoRA fine-tuning, and nonlinear heads trained on CLIP embeddings. For comparison, we also evaluate linear probes and MLPs trained on DINOv3, utilizing the largest and most recent version of the model.

Evaluation protocol: We adopt standard top-k recognition metrics (Top-1, Top-2, and Top-3 accuracy) to capture ranking quality, and we report Macro-F1 to account for class imbalance by averaging F1-scores across all classes. To obtain a more granular view of model behavior, we also inspect the confusion matrix, which highlights systematic confusions between visually related styles.

Relation to prior work and dataset choices: Our pipeline and design choices are grounded in the original CLIP formulation [1], prompt learning with CoOp [2], and parameter-efficient adaptation via LoRA [4]. For task-specific context, we refer to previous work on architectural style recognition, in particular the ArchiStyles dataset [5], and the study by Xu *et al.* titled *Architectural Style Classification Using Multinomial Latent Logistic Regression* [6], which inspired our label space, dataset selection, and evaluation methodology.

II. Dataset

We initially conducted our zero-shot experiments on the *ArchiStyles* dataset [5], a pre-processed and balanced collection of **10,615** images categorized into 25 architectural styles. The official split includes **86%** for training (9,138 images), **9%** for validation (979), and **5%** for testing (498). The label space covers a wide range of architectural movements, including *Achaemenid, American Foursquare, American Craftsman, Ancient Egyptian, Art Deco, Art Nouveau, Baroque, Bauhaus, Beaux-Arts, Byzantine, Chicago School, Colonial, Deconstructivism, Edwardian, Georgian, Gothic, Greek Revival, International Style, Novelty, Palladian, Postmodern, Queen Anne, Romanesque, Russian Revival,* and *Tudor Revival*.

However, the dataset's preprocessing pipeline—mainly rotations, mild noise injection, and white padding—was not aligned with the **reference visual preprocessing of CLIP** [1]. As emphasized by the official CLIP implementations [7], [8], CLIP expects a specific image transform sequence: resizing to the model input resolution, center or random cropping, and normalization to CLIP's mean and

Corso di Laurea "Artificial Intelligence Engineering", UNIMORE. Email: 276275@studenti.unimore.it, 270975@studenti.unimore.it , 300432@studenti.unimore.it

standard deviation, typically with bicubic resampling. When CLIP is used as a frozen feature extractor, any *mismatch in the preprocessing* can introduce distribution shifts that degrade downstream classifiers. In other words, CLIP is robust but not invariant to transformations that alter semantic content or edge statistics.

To avoid this mismatch and to compare methods under conditions consistent with the CLIP training assumptions, we switched to a subset of the dataset introduced in *Architectural Style Classification Using Multinomial Latent Logistic Regression* (MLLR) [6]. This dataset has the same labels we mentioned before and it provided access to **raw, non-preprocessed images**—even if imbalanced—allowing us to apply a CLIP-consistent image processing pipeline and establish a fair baseline against a pre-CLIP approach from 2014.

In our experiments, we follow CLIP's original preprocessing pipeline for both training and evaluation images. We tested multiple resizing and padding strategies, including black padding, white padding, and aspect-ratio preservation. The best-performing method was a "stretch" transformation, where images were resized to a square resolution without preserving the original aspect ratio, using Lanczos resampling. This approach preserved sufficient structural detail for style recognition while maintaining compatibility with CLIP's feature extractor. To address class imbalance, we constructed balanced validation and test splits. Additionally, during the training of LoRA, MLP, and linear probes, we employed Inverse Class Frequency Weighting within the Cross-Entropy loss. Finally, confusion matrices are used to visualize systematic confusions between visually similar styles.

III. CLIP Zero-Shot Validation and Comparison

Benchmark scope and protocol: We evaluated zero-shot recognition using the same textual prompt template defined in the CLIP paper: *"A photo of a {label}"* The benchmark includes **73 pretrained models** spanning four major CLIP families:

- **OpenCLIP:** 34 variants trained on LAION, DataComp, or CLIPA datasets;
- **MetaCLIP:** 12 variants (e.g., worldwide and fullcc checkpoints);
- **SigLIP and SigLIP2:** 20 variants trained on the WebLI corpus, at resolutions 256, 378, 384, and 512;
- **OpenAI CLIP:** 7 official models, including ViT-B/32, ViT-B/16, ViT-L/14, ViT-L/14-336, RN50, RN101, and RN50x64.

Models differ in backbone scale (from ViT-B/32 to ViT-bigG-14 and from RN50 to RN50x64), input resolution (ranging from 224 to 512, with common settings at 336, 378, 384, and 448), and pre-training corpus (OpenAI's proprietary data, LAION/DataComp for OpenCLIP, MetaCLIP's curated sources, and WebLI for SigLIP2). We report **Top-1**, **Top-2**, and **Top-3** accuracy together with **Macro-F1**, and analyze confusion matrices to capture fine-grained errors between visually similar styles. All matrices are available on the GitHub repo.

Headline zero-shot results: Across all families, zero-shot CLIP models provided strong baselines for architectural style recognition. The highest overall accuracy was obtained by **SigLIP** (*ViT-SO400M-14-SigLIP-378*), reaching **Top-1 = 71.5**, **Top-3 = 90.3**, and **Macro-F1 = 69.3** (Tab. I). **MetaCLIP ViT-bigG-14 (worldwide)** followed closely with **Top-1 = 70.2**, the highest **Top-3 = 91.1**, and **Macro-F1 = 68.7**. The best **OpenCLIP** reference (*ViT-bigG-14-CLIPA, DC-1B*) reached **Top-1 = 66.0**, **Top-3 = 86.8**, and **Macro-F1 = 64.3**. Among OpenAI checkpoints, *RN50x64* provided **Top-1 = 57.7**. Overall, OpenCLIP models offered a solid compromise between accuracy (e.g., 66.0% Top-1 for ViT-bigG-14-CLIPA) and full public availability, whereas OpenAI CLIP checkpoints supplied stable and well-documented "size ladders" (e.g., ViT-B/32 $\rightarrow$ ViT-L/14 $\rightarrow$ RN50x64), which we use as controlled baselines in subsequent experiments, also given their frequent citation in the literature.

Implementation details: We strictly follow the official CLIP inference pipeline, as described in [1] and the reference implementation guideline provided by OpenAI. All models are evaluated using the same prompt template ensuring fair and reproducible comparisons across architectures. As a protocol note, the results reported in Table I (and the two accompanying checks mentioned in this section) are computed in *zero-shot* on the *training split* (no fine-tuning), which is used solely as the evaluation set for these sanity checks.

IV. Prompt Ensemble

Following the idea introduced in *Aggregate-and-Adapt Natural Language Prompts for Downstream Generalization of CLIP* [9], and a broad body of related work, our second approach employs a **prompt ensemble**. Instead of relying on a single textual description per class, we evaluate multiple semantically equivalent prompts and aggregate their scores. This strategy improves zero-shot robustness by reducing the variance caused by any single wording.

We optimized a small set of class-agnostic templates through a few iterations and applied them uniformly to all classes, where the class label fills the curly brackets. The full list of templates is as follows:

"a photo of {label} architecture",
"a photograph of {label} architecture",
"an image of {label} architecture",
"a picture of {label} architecture",
"a photo of the {label} architectural style",
"the {label} architectural style",
"a photo of a {label} style building",
"a {label} style building",
"a building in the {label} style",
"{label}-style building",

TABLE I

ZERO-SHOT RESULTS FOR REFERENCE CLIP-LIKE MODELS (TRAIN SPLIT).

Family	Models	Pretraining	Top-1(%)	Top-2(%)	Top-3(%)	Macro-F1(%)	number parameters ViT
MetaCLIP	ViT-bigG-14 (worldwide)	curated (worldwide)	70.2	85.2	91.1	68.7	1.8B
SigLIP	ViT-SO400M-14-SigLIP-378	WebLI	71.5	84.8	90.3	69.3	428.2M
OpenCLIP	ViT-bigG-14-CLIPA (DC-1B)	DataComp-1B	66.0	80.0	86.8	64.3	1.8B
OpenAI	ViT-L-14	proprietary	54.8	69.8	78.2	53.4	304.0M
OpenAI	ViT-B-32	proprietary	51.3	68.0	78.9	48.5	87.8M
OpenAI	RN50x64	proprietary	57.7	74.7	81.7	55.9	420.8M

```
# image_encoder - Vision Transformer or ResNet backbone
# text_encoder  - Text Transformer
# I[n, h, w, c] - minibatch of input images
# P[c, t]       - prompt templates per class (classes,
    templates)
# W_i, W_t      - learned projections to embedding space
# tau           - model temperature

# Step 1: Encode image batch
I_f = image_encoder(I)
I_e = l2_normalize(np.dot(I_f, W_i), axis=1)

# Step 2: Encode text prompts for each class
T_f = []
for c in range(C):
    prompts = P[c]
    T_c = text_encoder(prompts)
    T_c_e = l2_normalize(np.dot(T_c, W_t), axis=1)
    T_f.append(T_c_e)

# Step 3: Compute ensemble logits per class
class_logits = []
for c in range(C):
    logits_c = np.dot(I_e, T_f[c].T) * np.exp(tau)
    logits_c_mean = np.mean(logits_c, axis=1)
    class_logits.append(logits_c_mean)

# Step 4: Stack logits and compute predictions
logits = np.stack(class_logits, axis=1)
probs = softmax(logits, axis=1)
preds = argmax(probs, axis=1)
```

Code 1. Prompt Ensamble approach.

"a {label} building",
"a building of {label} architecture",
"a representation of {label} architecture",
"the facade of a {label} building",
"an example of {label} architecture".

We follow the CLIP scoring convention: the cosine similarity between the image embedding and each text embedding, scaled by the model temperature. For each class, we average the logits across all prompts and apply the softmax operation once at the end. Averaging logits—rather than probabilities—maintains temperature calibration and is the common choice in the literature. We did not employ class-wise reweighting or heuristic prompt selection.

Our pseudo-code (Algorithm 1) outlines the ensemble evaluation process for an arbitrary CLIP-like model.

We applied the same 15-prompt ensemble to all **73** models described in Section 3.2 (OpenCLIP, MetaCLIP, SigLIP2, and OpenAI CLIP), maintaining identical preprocessing, data splits, and evaluation metrics. Table II summarizes zero-shot results (Top-1/2/3 accuracy and Macro-F1) for the **Zero-shot (train)** scenario.

Headline zero-shot results (train).:

- **Best overall:** MetaCLIP2 *ViT-bigG-14-worldwide* — Top-1: **0.745**, Top-2: 0.880, Top-3: 0.930, Macro-F1: 0.731.
- **Best SigLIP:** *SigLip2 ViT-gopt-16-SigLIP2-384* — Top-1: **0.730**, Top-2: 0.865, Top-3: 0.923, Macro-F1: 0.720.
- **Best OpenCLIP:** *ViTamin-XL-336* (DataComp-1B) — Top-1: **0.7063**, Top-2: 0.8492, Top-3: 0.9022, Macro-F1: 0.6889.
- **Best OpenAI CLIP:** *RN50x64* — Top-1: **0.6325**, Top-2: 0.7844, Top-3: 0.8604, Macro-F1: 0.6222.

Despite its minimal implementation effort, zero-shot classification with prompt ensemble achieves significantly improved performance, making it a highly effective strategy even without fine-tuning.

Reference models for this study: To keep the paper focused and reproducible, we worked on the best model of MetaCLIP2 and a consistent OpenAI CLIP size progression for comparison. In the rest of the paper, we rely mainly on OpenCLIP ViT checkpoints due to their accessibility and variety, while including the top MetaCLIP model as the strongest zero-shot prompt ensamble reference point.

V. VALIDATING CLIP FOR OUR TASK

Before moving forward, we validated our choice to perform classification with CLIP-based models. To justify this decision, we compared CLIP against finetuned non-contrastive baselines, including YOLOv11x adapted for image classification and a modern ConvNeXtV2.

We intentionally selected a low-effort CLIP pipeline (prompt-based inference with minimal engineering) to keep the comparison fair and practical.

All methods were evaluated on the same test split. As reported in Table III, even if ConvNeXtV2 reached better overall accuracy, our zero-shot CLIP models achieved performance comparable to fine-tuned CNN-based models. These outcomes confirmed that CLIP is a suitable approach for our setting and motivated us to continue with CLIP-centric experiments in the rest of the study.

TABLE II

RESULTS FOR ZERO-SHOT WITH PROMPT ENSAMPLE (TRAIN SPLIT).

Checkpoint	Pretraining Corpus	Top-1 (%)	Top-2 (%)	Top-3 (%)	Macro-F1 (%)	Δ Top-1 vs Tab. I (%)
ViT-bigG-14-worldwide	metaclip2_worldwide	74.5	88.0	93.1	73.3	+4.3
ViT-gopt-16-SigLIP2-384	webli	73.1	86.5	92.3	72.0	+4.2
ViT-L-14	openai	61.2	77.1	83.8	60.6	+6.4
ViT-B-32	openai	55.2	73.2	82.5	54.7	+3.9

TABLE III

ZERO-SHOT AND PROMPT ENSAMPLE (TEST) COMPARISON OF BASELINES. ROWS ARE SORTED BY INCREASING TOP-1.

Model	Top-1(%)	Top-3(%)	Macro-F1(%)
ConVneXtV2	80.4	95.7	80.2
YOLOv11x-cls	74.9	92.7	74.6
ViT-bigG-14-worldwide	73.1	92.8	72.6
ViT-L-14 (OpenAI)	61.9	84.1	60.5
ViT-B-32 (OpenAI)	59.5	81.7	57.8

VI. LINEAR PROBE

Motivation: As shown in CLIP [1], contrastive visual features are often linearly separable. This means that a simple classifier on top of frozen embeddings can already deliver strong accuracy. We therefore study a **linear probe**: the image encoder is frozen and we train a single linear layer on the training set. The aim is to measure how much performance we can gain with minimal supervision and without the cost and instability of full fine-tuning.

Introduction to DINO

To contextualize CLIP's embedding quality, we also tried probes on features extracted from **DINOv3** [10], a self-supervised Vision Transformer model trained using the self-distillation with no labels. DINO provides high-quality visual features that are fully unimodal (vision-only), allowing a fair comparison against CLIP's multimodal embeddings. This helps us understand whether CLIP, which learns from both images and text, or DINO, which learns only from images, produces features that make it easier to tell architectural styles apart using a simple classifier.

Setup: We extract features from several pretrained encoders and train a linear classifier. Hyperparameters (learning rate, weight decay, batch size) are reported per model in Table IV. All results below are computed on the test split.

Implementation details: In Algorithm 2 we show our minimal linear-probe loop: We freeze the image encoder, extract normalized embeddings, and train a single linear classifier with cross-entropy and ℓ_2 regularization.

Comparison to zero-shot baselines: For the same encoders evaluated in zero-shot (second screenshot), the linear probe brings large gains:

```
# Setup encoder (frozen) and preprocessing
e = image_encoder(pretrained=True, frozen=True)
p = enc.preprocessing

# Extract normalized embeddings
Z_train, y_train = encode_dataset(train_split(X, y), e, p)
Z_val,   y_val   = encode_dataset(val_split(X, y),   e, p)

# Initialize linear classifier
W = randn(d, C) * 0.01
b = zeros(C)

# Training loop
for epoch in range(E):
  for Z_b, y_b in iterate_minibatches(Z_train, y_train, B)
      :
    logits = Z_b @ W + b
    loss = cross_entropy(softmax(logits), y_b) * l2_norm(W
        )
    dW, db = grad(loss, [W, b])
    W, b = opt.update([W, b], [dW, db])

  # Validation
  val_acc = accuracy(argmax(Z_val @ W + b), y_val)

# Inference
def predict(x):
  z = normalize(enc(preproc(x)))
  return argmax(z @ W + b)
```

Code 2. Linear Probe approach

Discussion: These results align with prior literature: linear probing is a robust and inexpensive way to unlock most of the value of high-quality visual encoders. CLIP features are known to be linearly separable for many tasks [1], so even a single fully connected layer can exploit label supervision effectively. Similarly, self-supervised ViT encoders like DINO and DINOv2 report strong linear-probe performance across diverse datasets [11], [12]. Our findings match this trend: compared with zero-shot, linear probing adds **+22–24 percentage points** Top-1 for OpenAI ViT-B/32 and ViT-L/14, and yields very high Macro-F1, indicating improvements that are not limited to the most frequent classes.

VII. CLIP WITH MLP

Motivation: After the linear probe results, a natural question is whether a *nonlinear* classifier on top of CLIP features can do better. Although we knew from CLIP paper that embeddings are often linearly separable [1], a shallow MLP may capture residual nonlinearity between classes and yield extra accuracy, while keeping the backbone frozen and training cost low.

TABLE IV

LINEAR PROBE RESULTS ON THE TEST SPLIT (FROM THE FIRST SCREENSHOT).

Model	Top-1	Top-2	Top-3	Macro-F1	Δ zero-shot	Δ prompt ens.	weight	
ViT-B-32, openai	73.8	87.9	93.6	73.6	22.5	18.6	87.8	M
ViT-L-14, openai	82.2	92.7	95.9	82.1	27.3	21.0	304.0	M
ViT-bigG-14-worldwide, metaclip2_worldwide	85.9	96.1	98.2	85.9	15.7	11.0	1.8	B
DINOv3-ViT_7b16 (pretrain-lvd1689m)	87.5	95.6	98.3	87.5	n/a	n/a	6.7	B

```
# Setup encoder CLIP (frozen) e preprocessing
enc  = image_encoder(CLIP_name, pretrained=True, frozen=
     True)
prep = enc.preprocessing
mlp  = MLP(hidden_layers=[(h1, ReLU), (h2, ReLU), ...],..)

# Extracting e caching degli embedding
# Z_train, y_train
# Z_val,   y_val

# Training loop del MLP
for epoch in range(E):
  for Z_b, y_b in minibatches(Z_train, y_train, B,):
    logits = mlp(Z_b)
    loss = cross_entropy(logits, y_b)
    opt.zero_grad()
    loss.backward()
    opt.step()
  if sched: sched.step()

  # Validation
  val_logits = mlp(Z_val)
  val_preds  = argmax(val_logits, axis=1)
  val_acc    = accuracy(val_preds, y_val)
  update_best_state_if(val_acc)

# Inference
def predict(x):
  z = enc(prep(x))
  return argmax(mlp(z))
```

Code 3. MLP

Method: We freeze the CLIP visual encoder and cache its ℓ_2-normalized embeddings for the train/validation splits. On top of these features, we train a shallow MLP with L hidden layers, ReLU (or GELU) activations, dropout, and a final linear layer to our C classes. We optimize weighted cross-entropy with AdamW and apply weight decay. We used a cosine learning rate scheduler and we monitor validation Top-1 (or Macro-F1) for early stopping.

Implementation details: Algorithm 4 shows the minimal training loop on frozen CLIP embeddings.

Results and comparison: We decided to compare the MLP head with zero-shot CLIP (single prompt) and with a linear probe, using the same test split to show the differences between these approches. On **ViT-B/32**, the MLP reaches **77.9%** Top-1 and **77.8%** Macro-F1, outperforming the linear probe by **+4.1/+4.1** points and the zero-shot single/ensemble by **+26.6/+22.7** (Top-1) and **+29.3/+23.1** (Macro-F1).
On **ViT-L/14**, the MLP attains **84.3%** Top-1 and **84.2%** Macro-F1, yielding **+2.1/+2.1** over the linear probe and **+29.5/+23.1** (Top-1) and **+30.8/+23.6** (Macro-F1) over zero-shot single/ensemble.
For **MetaCLIP ViT-bigG-14**, MLP improves to **87.1/87.0** (Top-1/Macro-F1), i.e., **+1.1/+1.1** vs. the linear probe and **+16.82/+18.3** vs. zero-shot (single); gains remain **+11.8/+12.5** over the prompt ensemble.
Finally, with **DINOv3-ViT-7b16** the MLP matches linear-probe Top-1 (**87.5%**) and is essentially on par in Macro-F1 (**-0.04**), **confirming that strong unimodal features are already near linearly separable.** In fact, for the smaller benchmark models we used (ViT-B/32 and ViT-L/14), it can be assumed that lower-quality features are better classified by the MLP approach we just applied.
Overall, a shallow nonlinear head delivers consistent gains over linear probes for CLIP backbones and achieves our best score up to this point on contrastively trained vision encoders (MetaCLIP), while DINOv3 remains the top performer on this dataset.

VIII. CoOp Prompt Learning

Motivation: Since Zero-shot classification is done by encoding text prompts such as "a photo of a {class}" for all classes and picking the nearest label to an image embedding, This process is simple but sensitive to the prompt design. Literature gives to us CoOP method that replace the fixed textual context with learnable tokens that are optimized on a small training set. In this section our goal is to implement CoOP and quantify its benefit over zero-shot CLIP our dataset.

Setup: We use OpenCLIP with ViT-B/32 and ViT-L/14 backbones. The image encoder is frozen during CoOP training; only the context tokens are learned.

Implementation details: We use the template *"a photo of a {}"*. Let C be the number of classes and D the text embedding dimension. CoOP learns n_{ctx} continuous tokens (we set n_{ctx}=16). We study two modes:

- **Shared**: a single context matrix $\mathbf{U} \in \mathbb{R}^{n_{\text{ctx}} \times D}$ is shared by all classes.
- **Per-class**: a context tensor $\mathbf{U} \in \mathbb{R}^{C \times n_{\text{ctx}} \times D}$ is learned, one set per class.

The learned tokens replace placeholders right after the BOS token. The resulting embeddings are passed through the CLIP text transformer; logits are computed as scaled cosine similarities between image features and the text features for all classes.

Method: We optimize the context parameters with AdamW (learning rate 10^{-3}, weight decay 0) for 10 epochs. We freeze all CLIP parameters and only update the context

TABLE V

ZERO-SHOT VS LINEAR PROBE VS CLIP + MLP (TOP-1 / MACRO-F1, %). Δ COLUMNS SHOW ABSOLUTE MLP GAINS OVER THE CORRESPONDING BASELINE. "N/A" MEANS NOT APPLICABLE (NO ZERO-SHOT FOR DINO).

Model	**Zero-shot (single)**		**Linear probe**		**MLP (ours)**		**Δ MLP − single**		**Δ MLP − probe**	
	Top-1	**Macro-F1**	**Top-1**	**Macro-F1**	**Top-1**	**Macro-F1**	**Top-1**	**Macro-F1**	**Top-1**	**Macro-F1**
ViT-B/32 (OpenAI)	51.3	48.5	73.8	73.6	77.9	77.8	+26.6	+29.3	+4.1	+4.1
ViT-L/14 (OpenAI)	54.8	53.3	82.2	82.1	84.3	84.2	+29.5	+30.8	+2.1	+2.1
ViT-bigG/14 (MetaCLIP)	70.2	68.7	85.9	85.9	87.0	87.0	+16.8	+18.3	+1.1	+1.1
DINOv3-ViT-7b16	N/A	N/A	87.5	87.5	87.5	87.4	N/A	N/A	+0.0	+0.0

tokens.

As before we report Top-1 accuracy and Macro-F1 on the test set. For the zero-shot comparisons, we evaluate: (a) a single handcrafted prompt, and (b) a empty prompt made up only by the *"{}"* class label. The best CoOP results we obtained are:

- ViT-B/32: 69.6 % Top-1, 69.6 % Macro-F1.
- ViT-L/14: 75.4 % Top-1, 75.3 % Macro-F1.

Results

Table VI summarizes the results and shows absolute improvements (Δ in percentage points) over both zero-shot baselines. CoOP gives large gains for both architectures, and the improvement remains strong. Although the results obtained with the CoOP approach exceeded expectations and are comparable to the gains achieved with the MLP-based classifier (e.g., ViT-B/32 reached a Top-1 improvement of +18.3 with CoOP versus +22.5 with MLP), we were unable to apply CoOP to the larger ViT-bigG/14 (MetaCLIP) model due to limited computational resources. As a result, comparative results for this backbone are not available. Furthermore, based on the performance observed and guided by insights from the literature, we decided not to implement CoCoOp in our experiments. While CoCoOp is known to perform better on "novel" classes—potentially relevant to our dataset, where architectural styles may be visually similar and not easily separable—we opted to focus our efforts on the LoRA-based approach. Given the relatively small size of our dataset, we prioritized methods that offered better scalability and efficiency under constrained training conditions.

Discussion

Although we implemented both the Shared Context and Class-Specific Context (CSC) strategies, our experiments revealed negligible performance differences between the two modes. We hypothesize that this convergence is driven by two primary factors:

1. Overfitting in Low-Data Regimes: The Class-Specific mode introduces a significantly larger number of learnable parameters, as it optimizes independent context vectors for each class. Given the limited size of our training split, this increased dimensionality likely caused the model to memorize training noise rather than learning robust, generalizable features. In contrast, the Shared Context mode, having fewer degrees of freedom, acts as a constrained model that naturally regularizes the training, leading to better generalization on unseen data.

2. The Visual Bottleneck: Our dataset involves fine-grained architectural distinctions where inter-class similarity is high (e.g., distinguishing Baroque from Rococo). Since the image encoder remains frozen throughout the CoOp training, the visual embeddings are fixed. If the pre-trained CLIP backbone cannot inherently resolve the subtle visual differences between these styles in its embedding space, increasing the complexity of the textual prompts via class-specific tokens yields diminishing returns. The limitation, in this case, lies in the fixed visual representation rather than the textual alignment."

```
# Input:
#   Y: Set of class labels
#   W_e: Pre-trained fixed embedding matrix (Frozen)
#   V_ctx: Learnable context vect [Parameter, shape M x D]

def Forward_PromptLearner():
    P_batch = []

    for y_k in Y:
        # Create raw text with placeholders
        # text_raw = "[SOS]" + ("[CTX]" * M) + "a photo of
              a " + y_k + "[EOS]"

        idxs = tokenizer(text_raw)  # [L]

        # Look up fixed embeddings (No Gradients)
        e_fixed = W_e[idxs]  # [L, D]

        # 2. Vector Injection:  Overwrite positions 1 to M
              with learnable vectors
        e_mixed[1 : M+1] =  V_ctx # Index 0 is [SOS]

        P_batch.append(e_mixed)

    # Stack into a single batch tensor
    E_prompts = stack(P_batch)  # Shape [K, L, D]

    return E_prompts
```

Code 4. CoOp: how text transformer input embedding is made with context tokens.

IX. LORA

Motivation: From the literature, we learned that it is possible to fine-tune the CLIP architecture, we choose the one called LoRA, which stands for Low-Rank Adaptation [13], [3], so our forward step was to apply LoRA to the CLIP visual encoder in order to enable parameter-efficient fine-tuning. Instead of updating all weights, we insert small

TABLE VI

COMPARISON BETWEEN ZERO-SHOT CLIP AND COOP ON OUR DATASET. WE REPORT TOP-1 AND MACRO-F1 (%). Δ VALUES ARE ABSOLUTE GAINS IN PERCENTAGE POINTS.

Model	**Zero-shot (single)**		**Zero-shot (ensemble)**		**CoOP**		**Δ vs single**		**Δ vs ensemble**	
	Top-1	**Macro-F1**	**Top-1**	**Macro-F1**	**Top-1**	**Macro-F1**	**Top-1**	**Macro-F1**	**Top-1**	**Macro-F1**
ViT-B/32	51.3	48.5	55.2	54.7	69.6	69.6	+18.3	+21.1	+14.4	+14.9
ViT-L/14	54.8	53.4	61.2	60.6	75.4	75.3	+20.6	+21.9	+14.3	+14.7

low-rank adapters into selected *linear layers* of the vision transformer and train only those adapters. This reduces the number of trainable parameters and allows us to deal with our limited dataset while preserving the stability of the frozen backbone [13], [3].

Setup: We begin with OpenCLIP checkpoints and use the original CLIP preprocessing and normalization. Unless stated otherwise, the image encoder is kept frozen, and only the LoRA parameters and the small classification head are updated during training. As before we target our two CLIP ViT backbones (ViT-B/32 and ViT-L/14) to study the effect of model scale. Also metrics follow the previous sections: Top-1/Top-2/Top-3 accuracy and Macro-F1, with a confusion matrix for error analysis.

Implementation Details: **Where we attach LoRA.** We traverse the CLIP visual transformer and wrap every `nn.Linear` whose qualified name matches a target in {`out_proj`, `c_fc`, `c_proj`}. These correspond to the attention output projection and the MLP projections of all the encoders blocks.

Results and Comparison: We evaluate the performance of LoRA-enhanced models using the same data splits and preprocessing protocols adopted for zero-shot, prompt ensembling, and linear probing benchmarks. To gain a deeper understanding of the learned representations, we utilize the LoRA-fine-tuned encoder as a backbone, attaching both a Linear Probe and a Multi-Layer Perceptron (MLP) as classification heads.

The results, summarized in Table VII, reveal that the choice between an MLP and a simple Linear Probe does not lead to a significant divergence in performance. For instance, on the ViT-L/14 backbone, both classifiers achieve a comparable Top-1 accuracy (85.5 %). This observation suggests that the fine-tuning process yields a highly optimized embedding space where class boundaries are effectively linearized, rendering the additional non-linear depth of an MLP largely redundant.

Furthermore, we observe that the benefits of LoRA are particularly pronounced as the model scale increases. Larger architectures, such as the ViT-bigG-14, exhibit consistent improvements over their frozen-feature counterparts, reaching the highest overall performance of 88.0 %. This trend can be attributed to the higher number of transformer blocks available in larger models: since LoRA adapters are injected into each block, the cumulative capacity of the trainable parameters scales with the backbone size, allowing the model to better capture the complex visual nuances of our dataset.

Ultimately, LoRA bridges the gap between frozen-feature extraction and full fine-tuning. It provides a robust accuracy–efficiency trade-off, maintaining the stability of the original backbone while achieving competitive results—surpassing even the strong DINOv3 baseline (87.5%)—with a significantly lower memory footprint during training.

TABLE VII

IMPACT OF THE CLASSIFICATION HEAD ON LORA-FINETUNED BACKBONES. THE SIMPLE LINEAR PROBE ACHIEVES COMPARABLE OR SUPERIOR PERFORMANCE TO THE MLP, SUGGESTING THAT LORA OPTIMIZATION YIELDS A LINEARLY SEPARABLE EMBEDDING SPACE. (%)

Backbone (LoRA)	**Classifier**	**Top-1**	**Top-3**	**F1-Macro**
ViT-B/32	Linear Probe	77.12	93.75	76.94
	MLP	76.70	93.47	76.53
ViT-L/14	Linear Probe	85.47	96.63	85.44
	MLP	85.47	93.75	85.40
ViT-bigG-14	Linear Probe	88.00	97.89	87.98
	MLP	87.58	97.12	87.53

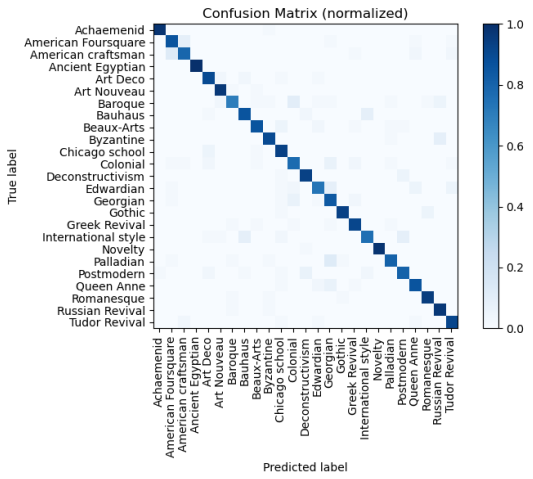


Fig. 1. Confusion Matrix of our best performance methods LoRa.

X. CONCLUSION

In this study, we presented a systematic benchmark of CLIP-based strategies for architectural style classification, ranging from zero-shot inference to parameter-efficient fine-tuning. Our results demonstrate that while Zero-Shot CLIP and Prompt Ensembling provide robust baselines, significant

performance gains can be unlocked with minimal supervision. Specifically, training a non-linear classifier (MLP) on frozen embeddings proved to be a highly efficient strategy, offering a better trade-off than simple Linear Probes or CoOp on smaller backbones.

However, our experiments identify Low-Rank Adaptation (LoRA) as the superior approach. By fine-tuning a small fraction of parameters, LoRA effectively bridges the gap between frozen features and full training, allowing the largest OpenCLIP models to surpass strong unimodal baselines like DINOv3. We conclude that for limited datasets in the architectural domain, coupling a frozen encoder with an MLP is the most cost-effective solution, while LoRA is the recommended choice when maximizing accuracy is paramount

REFERENCES

[1] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever, "Learning transferable visual models from natural language supervision," in *Proceedings of the International Conference on Machine Learning (ICML)*, 2021. [Online]. Available: https://arxiv.org/abs/2103.00020

[2] K. Zhou, J. Yang, C. C. Loy, and Z. Liu, "Learning to prompt for vision-language models," *International Journal of Computer Vision (IJCV)*, 2022.

[3] M. Zanella and I. Ben Ayed, "Low-rank few-shot adaptation of vision-language models," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Workshops*, 2023. [Online]. Available: https://arxiv.org/abs/2303.11367

[4] Y. Chen *et al.*, "Low-rank few-shot adaptation of vision-language models," https://arxiv.org/pdf/2405.18541, 2024, arXiv preprint arXiv:2405.18541.

[5] M. R. Team, "Archistyles dataset," https://universe.roboflow.com/mipt4/archistyles, 2023, accessed: October 2025.

[6] Z. Xu, D. Tao, Y. Zhang, J. Wu, and A. C. Tsoi, "Architectural style classification using multinomial latent logistic regression," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*. IEEE, 2014, pp. 688–695.

[7] OpenAI, "Interacting with clip," https://colab.research.google.com/github/openai/clip/blob/master/notebooks/Interacting_with_CLIP.ipynb, 2021, accessed: October 2025.

[8] H. Face, "Clip feature extractor — hugging face transformers," https://huggingface.co/transformers/v4.7.0/_modules/transformers/models/clip/feature_extraction_clip.html, 2021, accessed: October 2025.

[9] C. Huang, S. Seto, S. Abnar, D. Grangier, N. Jaitly, and J. Susskind, "Aggregate-and-adapt natural language prompts for downstream generalization of clip," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. Apple Machine Intelligence Research, 2023. [Online]. Available: https://arxiv.org/abs/2303.12009

[10] O. Siméoni, H. V. Vo, M. Seitzer, F. Baldassarre, M. Oquab, C. Jose, V. Khalidov, M. Szafraniec, S. Yi, M. Ramamonjisoa, F. Massa, D. Haziza, L. Wehrstedt, J. Wang, T. Darcet, T. Moutakanni, L. Sentana, C. Roberts, A. Vedaldi, J. Tolan, J. Brandt, C. Couprie, J. Mairal, H. Jégou, P. Labatut, and P. Bojanowski, "Dinov3," https://arxiv.org/abs/2508.10104, 2025, arXiv preprint arXiv:2508.10104.

[11] M. Caron *et al.*, "Emerging properties in self-supervised vision transformers," *arXiv preprint arXiv:2104.14294*, 2021.

[12] M. Oquab, T. Darcet, T. Moutakanni, H. V. Vo, M. Szafraniec, V. Khalidov, P. Fernandez, D. Haziza, F. Massa, A. El-Nouby, M. Assran, N. Ballas, W. Galuba, R. Howes, P.-Y. Huang, S.-W. Li, I. Misra, M. Rabbat, V. Sharma, G. Synnaeve, H. Xu, H. Jegou, J. Mairal, P. Labatut, A. Joulin, and P. Bojanowski, "Dinov2: Learning robust visual features without supervision," https://arxiv.org/abs/2304.07193, 2023, arXiv preprint arXiv:2304.07193.

[13] E. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, and W. Chen, "Lora: Low-rank adaptation of large language models," in *International Conference on Learning Representations (ICLR)*, 2022. [Online]. Available: https://arxiv.org/abs/2106.09685